



UANL

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN



FIME

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN
FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

“A3 - PIA - Proyecto final”

Redes Neuronales Artificiales

M.I.I. Blanca Idalia Pérez Pérez

Grupo: 004

Salón: 1201

MATRICULA	NOMBRE	PE	Hora
1952947	Heidi Pamela Martínez Martínez	ITS	V4
2077884	Melida Krystal Mendoza Ventura	ITS	N1
2109542	Marco Antonio Arreola de León	ITS	N1

Ciudad Universitaria, San Nicolás de los Garza, Nuevo León a 20 de mayo de
2025

Contenido

Introducción.....	3
Propuesta de proyecto final	4
Objetivo	4
Descripción Del Problema	4
Requerimientos Generales	4
Información para utilizar.....	5
Detalles de la información	6
Tipo de Información	6
Fuente de Datos.....	6
Clasificación y Adecuación	7
Justificación.....	7
Desarrollo.....	8
WBS	8
Timeline	10
Conclusión	22
Bibliografías.....	23

Introducción

En los primeros años de formación educativa, el aprendizaje de la escritura representa una habilidad fundamental en el desarrollo cognitivo, lingüístico y motor de los niños. Sin embargo, es común que durante este proceso se presenten dificultades en la correcta formación de letras y números, especialmente en edades tempranas. Factores como la coordinación, la percepción visual, el seguimiento de instrucciones o incluso la falta de motivación pueden influir negativamente en la calidad del aprendizaje, lo que conlleva a errores persistentes que dificultan la legibilidad y comprensión del texto escrito.

Normalmente, la corrección de estos errores ha dependido solo de la observación directa de docentes o padres, lo que puede ser subjetivo, tardado o incluso desmotivador si no se maneja de forma paciente y constructiva. En este contexto, la incorporación de herramientas tecnológicas puede ofrecer una alternativa poderosa y complementaria para asistir en este proceso de aprendizaje. Al aprovechar la accesibilidad de plataformas digitales, inteligencia artificial y recursos interactivos, es posible crear entornos que estimulen positivamente a los niños a seguir practicando de manera autónoma y divertida.

Con respecto a esto surge el proyecto “Asistente Mágico de Escritura”, una aplicación educativa diseñada para funcionar como guía interactiva durante el aprendizaje de la escritura. Su objetivo principal es reconocer letras y números dibujados por los niños mediante modelos de reconocimiento visual entrenados con aprendizaje automático, clasificar su corrección según patrones establecidos y ofrecer retroalimentación inmediata a través de sonidos, animaciones y recompensas visuales.

El desarrollo del sistema se plantea con herramientas accesibles como Kotlin en Android Studio y Figma para la construcción de la interfaz gráfica de usuario. Además de evaluar y corregir, motiva y hace agradable el proceso, ofreciendo recompensas por logros, y sugerencias visuales de mejora, todo dentro de un entorno seguro y atractivo.

Este proyecto aporta un valor al proceso de enseñanza, sino que además introduce a los niños de forma sencilla y práctica en conceptos básicos de inteligencia artificial, permitiéndoles observar cómo una máquina puede aprender a reconocer patrones a partir de ejemplos reales. De este modo, el Asistente Mágico de Escritura se convierte en una herramienta transversal: educativa, tecnológica y formativa, alineada con los desafíos de la educación moderna en entornos digitales.

Propuesta de proyecto final

Objetivo

El presente proyecto, titulado “Asistente Mágico de Escritura”, tiene como finalidad el desarrollo e implementación de una aplicación interactiva basada en inteligencia artificial, orientada al apoyo en el proceso de enseñanza de la escritura en niños de nivel preescolar y primaria. Esta herramienta busca reconocer letras y números dibujados a mano por los usuarios, corregir errores frecuentes en su trazo, y brindar retroalimentación positiva mediante sonidos, animaciones y recompensas visuales, incentivando así el aprendizaje a través del juego.

Descripción Del Problema

Muchos niños pequeños tienen dificultades para escribir correctamente letras y números. A menudo cometen errores como no cerrar bien una “a” o confundir letras parecidas como la “B” y la “D”. Estos errores son normales, pero si no se corrigen a tiempo, pueden afectar su aprendizaje y hacer que se frustren o pierdan el interés por escribir.

Los maestros y padres pueden ayudar, pero no siempre tienen el tiempo o las herramientas necesarias para dar retroalimentación inmediata y constante. Además, los métodos tradicionales pueden no ser tan atractivos para los niños.

Por eso, se necesita una herramienta interactiva que ayude a los niños a practicar la escritura de forma divertida. Esta herramienta debe reconocer lo que dibujan, decirles si está bien o mal, y motivarlos con sonidos, estrellas o animaciones. Así, los niños podrán aprender mientras juegan y se sienten motivados a mejorar.

Requerimientos Generales

A continuación, se detallan los principales requerimientos generales del proyecto:

Usabilidad

- La aplicación debe contar con una interfaz intuitiva y amigable para niños de entre 4 y 8 años.
- El sistema debe proporcionar instrucciones claras mediante texto, audio o animaciones simples.
- Las interacciones deben ser visualmente atractivas y fáciles de ejecutar.

Portabilidad

- El desarrollo debe ser multiplataforma, basado en tecnologías web (Java, Kotlin, Figma).
- No debe requerir instalación de software adicional en el dispositivo del usuario final.

Rendimiento

- El reconocimiento de letras o números debe realizarse en un tiempo máximo de 2 segundos tras terminar el trazo.

Accesibilidad

- Se deben considerar aspectos de accesibilidad como uso de colores contrastantes, sonidos claros, y retroalimentación visual para niños con dificultades auditivas o visuales leves.

Mantenibilidad

- El código fuente debe estar bien documentado y estructurado, permitiendo su mantenimiento y actualización por terceros.

Seguridad y privacidad

- El sistema no debe recolectar ni almacenar datos personales de los usuarios.
- Si se usan imágenes o trazos personalizados, deberán mantenerse solo en memoria local del dispositivo.

Información para utilizar

En este proyecto se utilizará información generada directamente por niños en edad escolar (de aproximadamente 5 a 8 años), quienes son el público objetivo del *Asistente Mágico de Escritura*. Los datos que se recogen consisten en dibujos de letras y números hechos a mano por los niños, ya sea en papel (fotografiados y digitalizados) o directamente en una interfaz digital de dibujo.

La información recolectada tiene como objetivo alimentar y evaluar el sistema de reconocimiento desarrollado, permitiendo identificar errores comunes en la escritura infantil, como trazos incompletos, letras invertidas o mal cerradas.

Detalles de la información

- **Tipo de datos:** Imágenes de letras de un dataset.
- **Forma de recolección:**
 - A través de tabletas o dispositivos con pantalla táctil, donde los niños dibujan en una aplicación desarrollada con Kotlin.
 - Alternativamente, dibujos escaneados o fotografiados hechos en papel.
- **Propósito del uso:**
 - Entrenar un modelo de inteligencia artificial para reconocer patrones de escritura correctos e incorrectos.
 - Validar la efectividad del modelo con ejemplos reales del público meta.
- **Tratamiento ético:**
 - No se recolecta información personal.
 - Las imágenes se utilizan únicamente con fines educativos y de investigación dentro del alcance del proyecto.

Tipo de Información

Para entrenar y probar el modelo del *Asistente Mágico de Escritura*, se utilizarán imágenes de letras escritas a mano digitalizados. El objetivo es que el sistema aprenda a reconocer trazos correctos e incorrectos de caracteres, como una "a" mal cerrada o una "d" invertida, usando solo letras mayúsculas.

Fuente de Datos

Se utilizarán dos fuentes principales:

- **Datos de entrenamiento (referencia externa):**
 - Se usará el dataset **A-Z Handwritten Alphabets in .csv format** basado en **NIST**: https://huggingface.co/datasets/pittawat/letter_recognition
 - Ejemplos: dibujos de letras como "A", "B", "C", etc.

- **Datos de prueba (propios o simulados):**
 - Se crearán dibujos en una interfaz, imitando el trazo que haría un niño.
 - Estos datos serán usados para evaluar el desempeño real del modelo y ajustar su sensibilidad.

Clasificación y Adecuación

Los datos se clasificarán en dos grandes grupos:

- **Trazos Correctos:** Letras o números bien formados.
- **Trazos Incorrectos:** Letras con errores comunes

Cada imagen será etiquetada según esta clasificación, lo cual permite que la red neuronal aprenda a diferenciar entre una letra bien escrita y una con errores.

Justificación

Esta aplicación se justifica por su enfoque innovador en el proceso de enseñanza de la escritura en niños de nivel preescolar y primaria. Al integrar redes neuronales, permite reconocer letras escritas a mano, identificar errores comunes en el trazo y ofrecer retroalimentación inmediata. Su diseño interactivo con elementos lúdicos como sonidos, animaciones y recompensas visuales convierte el aprendizaje en una experiencia divertida y motivadora. Además, su accesibilidad y adaptabilidad la hacen una herramienta valiosa tanto en el entorno escolar como en el hogar, fortaleciendo el desarrollo cognitivo y motriz de los niños a través del juego.

Desarrollo

WBS

En la gestión de proyectos, contar con una estructura organizada del trabajo fue fundamental para garantizar su correcta ejecución. En nuestro caso, implementamos el Work Breakdown Structure (WBS) como una herramienta clave para dividir el proyecto en tareas más manejables, facilitando la planificación y administración de recursos.

Para el desarrollo del Asistente Mágico de Escritura, establecimos un esquema jerárquico en el que cada fase se desglosó en actividades específicas. Desde la investigación inicial y el desarrollo del modelo de inteligencia artificial hasta el diseño de la interfaz y la implementación, este esquema nos permitió visualizar con claridad qué se debía hacer, en qué orden y con qué nivel de detalle.

Al adoptar este enfoque estructurado, optimizamos la coordinación entre los equipos de trabajo y aseguramos un control eficiente del avance del proyecto, reduciendo riesgos y mejorando la toma de decisiones.

1. Investigación y Planificación

- 1.1 Definición de objetivos del proyecto
- 1.2 Identificación de requerimientos técnicos y de usuario
- 1.3 Análisis de datasets disponibles
- 1.4 Diseño del esquema de aprendizaje automático
- 1.5 Planificación del desarrollo y recursos

2. Desarrollo del Modelo de IA

- 2.1 Selección de dataset base (A-Z Handwritten Alphabets)
- 2.2 Preprocesamiento de datos y limpieza
- 2.3 Entrenamiento inicial del modelo de reconocimiento
- 2.4 Ajuste de hiperparámetros y optimización
- 2.5 Pruebas con datos simulados
- 2.6 Validación con usuarios reales

3. Desarrollo de Aplicación

- 3.1 Diseño de arquitectura del software
- 3.2 Desarrollo backend en Java/Kotlin
- 3.3 Implementación de la lógica de reconocimiento con la IA
- 3.4 Integración con bases de datos de entrenamiento
- 3.5 Creación de APIs y comunicación con frontend

4. Diseño e Implementación de la Interfaz de Usuario

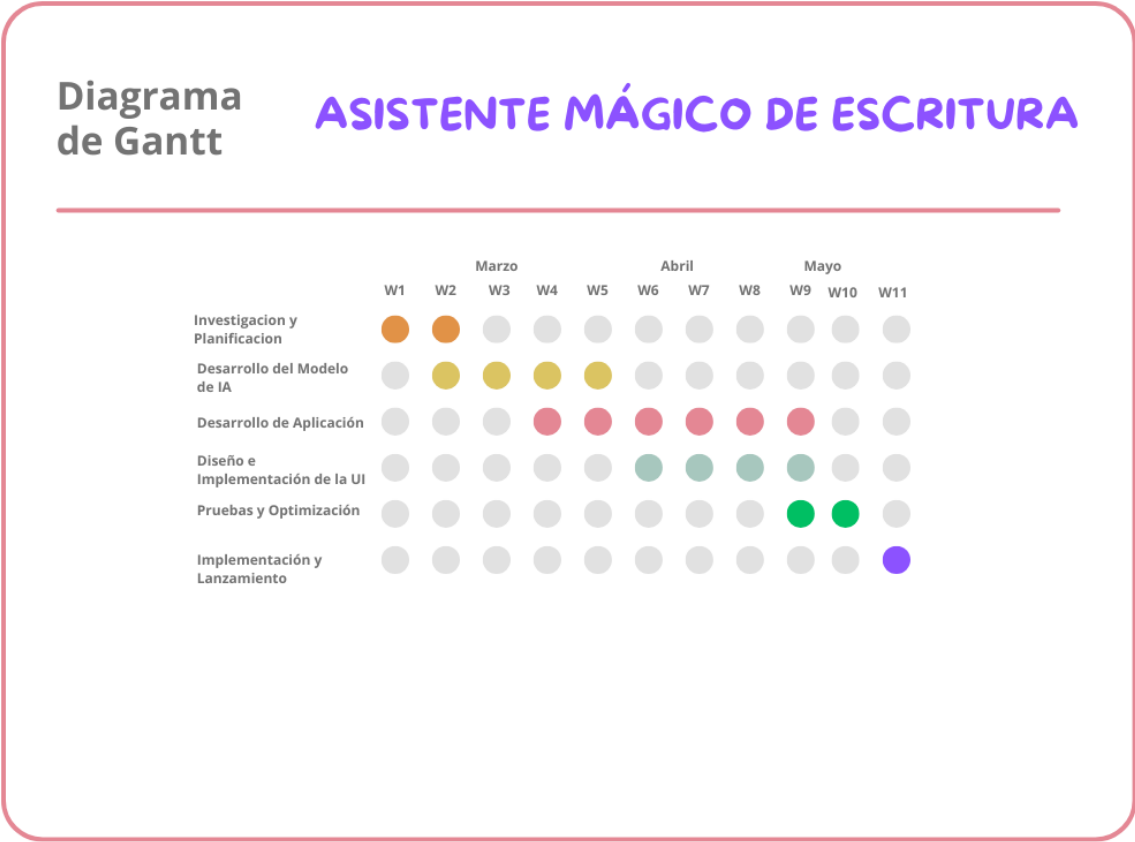
- 4.1 Diseño visual en Figma
- 4.2 Desarrollo de animaciones y retroalimentación interactiva
- 4.3 Implementación en Android Studio
- 4.4 Integración con funciones táctiles y de voz
- 4.5 Optimización para dispositivos móviles y tablets
- 4.6 Pruebas de usabilidad con niños

5. Pruebas y Optimización

- 5.1 Pruebas unitarias y de integración
- 5.2 Evaluación del modelo IA con nuevos datos
- 5.3 Ajuste de la interfaz y experiencia del usuario
- 5.4 Corrección de errores y optimización de rendimiento

6. Implementación y Lanzamiento

Timeline



Pantallas

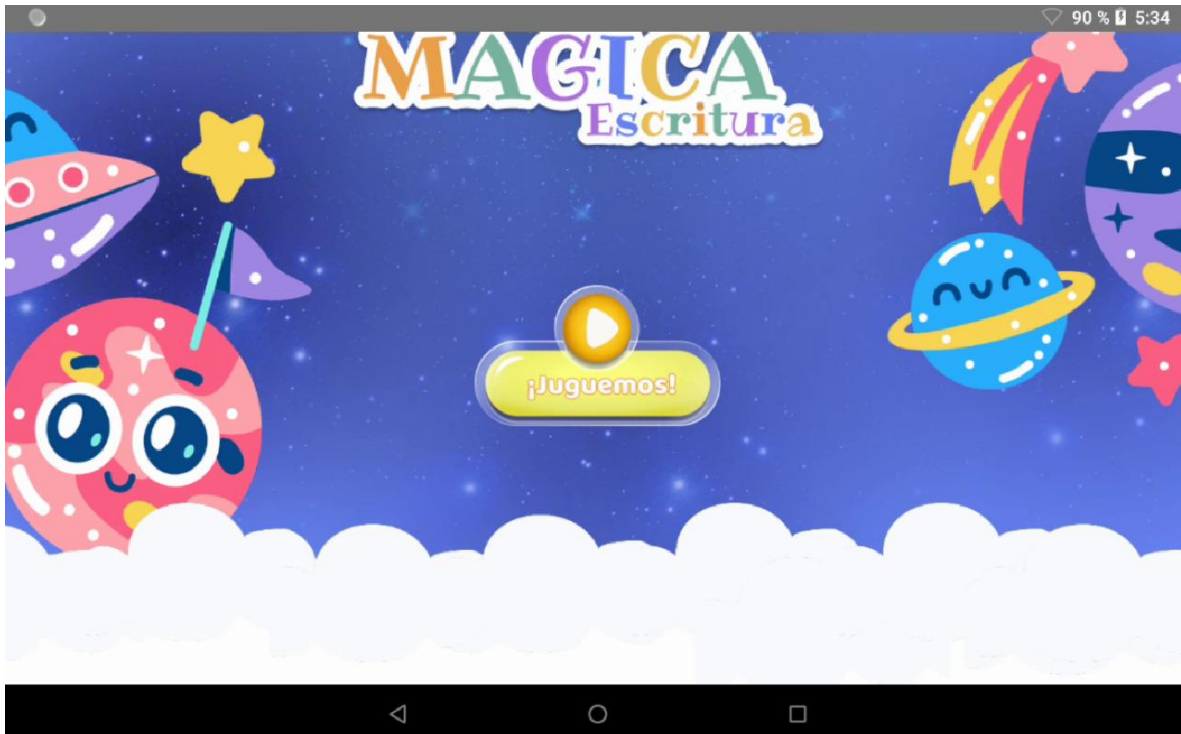


Imagen 1. Pantalla de inicio

```
package com.example.assistente_me.ui.screens

import androidx.compose.foundation.Image
import androidx.compose.foundation.layout.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.unit.dp
import com.example.assistente_me.R
import com.example.assistente_me.ui.components.ImageButton

@Composable
fun MainScreen(
    onJuguemosClick: () -> Unit,
) {
    Box(modifier = Modifier.fillMaxSize()) {
        // Fondo
        Image(
            painter = painterResource(id = R.drawable.background),
            contentDescription = "Fondo",
            contentScale = ContentScale.Crop,
            modifier = Modifier.fillMaxSize()
        )

        // Contenido principal
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(32.dp),
            verticalArrangement = Arrangement.Center,
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            ImageButton(onClick = onJuguemosClick)
        }
    }
}
```

```

        // Botón superior
        ImageButton(
            onClick = onJuguemosClick,
            imageRes = R.drawable.button1,
            modifier = Modifier.size(200.dp)
        )

        Spacer(modifier = Modifier.height(5.dp))

    }

}

}

```

Código 1. Pantalla de Inicio



Imagen 2. Pantalla de ejercicios

```

package com.example.assistente_me.ui.screens

import androidx.compose.foundation.gestures.detectDragGestures
import androidx.compose.ui.input.pointer.pointerInput
import androidx.compose.ui.graphics.Path
import androidx.compose.ui.unit.sp

import androidx.compose.foundation.Canvas
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*

```

```

import androidx.compose.material3.Button
import androidx.compose.material3.Text
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
import androidx.compose.ui.geometry.Offset
import androidx.compose.ui.graphics.Color as ComposeColor
import androidx.compose.ui.graphics.drawscope.Stroke
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.compose.ui.platform.LocalContext
import android.widget.Toast
import androidx.compose.foundation.Image
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.res.painterResource
import com.example.assistente_me.R

@Composable
fun DrawingScreen(
    onBack: () -> Unit,
    prediction: String,
    onPredict: (List<Offset>) -> Unit
) {
    Box(modifier = Modifier.fillMaxSize()) {
        Image(
            painter = painterResource(id = R.drawable.back2),
            contentDescription = "Fondo",
            contentScale = ContentScale.Crop,
            modifier = Modifier.fillMaxSize()
        )

        val path = remember { Path() }
        val points = remember { mutableStateListOf<Offset>() }
        var redrawTrigger by remember { mutableStateOf(0) }
        val context = LocalContext.current

        var letraObjetivo by remember {
            mutableStateOf(('A'..'Z').random().toString()) }

        Column(
            modifier = Modifier
                .fillMaxSize()
                .background(ComposeColor.Transparent)
                .padding(16.dp)
        ) {
            Spacer(modifier = Modifier.height(100.dp)) // Mueve el texto más
abajo

            Row(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(bottom = 8.dp),
                horizontalArrangement = Arrangement.Start
            ) {
                Text(
                    text = "Letra para dibujar: ",
                    color = ComposeColor.Black,
                    modifier = Modifier.alignByBaseline()
                )
                Text(
                    text = letraObjetivo,
                    color = ComposeColor.Black,
                    fontSize = 32.sp,
                    modifier = Modifier.alignByBaseline()
                )
            }
        }
    }
}

```

```

    )
}

Box(
    modifier = Modifier
        .weight(1f)
        .fillMaxWidth()
        .background(ComposeColor.Transparent)
) {
    Canvas(
        modifier = Modifier
            .fillMaxSize()
            .pointerInput(Unit) {
                detectDragGestures(
                    onDragStart = {
                        path.moveTo(it.x, it.y)
                        points.add(it)
                        redrawTrigger++
                    },
                    onDrag = { change, _ ->
                        path.lineTo(change.position.x,
change.position.y)

                        points.add(change.position)
                        change.consume()
                        redrawTrigger++
                    }
                )
            }
    ) {
        drawPath(path, color = ComposeColor.Black, style =
Stroke(width = 8f))
        val dummy = redrawTrigger
    }

    Spacer(modifier = Modifier.height(8.dp))

    Text(text = "Predicción: $prediction")

    Spacer(modifier = Modifier.height(8.dp))

    Row(
        modifier = Modifier.fillMaxWidth(),
        horizontalArrangement = Arrangement.SpaceEvenly
    ) {
        Button(onClick = {
            path.reset()
            points.clear()
            redrawTrigger++
            onPredict(emptyList())
        }) {
            Text("Borrar")
        }

        Button(onClick = {
            letraObjetivo = ('A'..'Z').random().toString()
            path.reset()
            points.clear()
            redrawTrigger++
            onPredict(emptyList())
        }) {
            Text("Nueva letra")
        }
    }
}

```

```

        Button(onClick = {
            if (points.isNotEmpty()) {
                onPredict(points.toList())
            } else {
                Toast.makeText(context, "Dibuja algo antes de
continuar", Toast.LENGTH_SHORT).show()
            }
        }) {
            Text(";Vamos!")
        }

        Spacer(modifier = Modifier.height(8.dp))

        Button(onClick = onBack, modifier = Modifier.fillMaxWidth()) {
            Text("Volver")
        }
    }
}

```

Código 2. Pantalla de Ejercicios



Imagen 3. Ejercicio contestado

2025-05-20 16:42:40.288 17319-17319 PredictionProb	com.example.assistente_me	D A: 0.38439083
2025-05-20 16:42:40.289 17319-17319 PredictionProb	com.example.assistente_me	D B: 0.24816148
2025-05-20 16:42:40.289 17319-17319 PredictionProb 2.7877046E-5	com.example.assistente_me	D C:
2025-05-20 16:42:40.289 17319-17319 PredictionProb 0.011291166	com.example.assistente_me	D D:
2025-05-20 16:42:40.289 17319-17319 PredictionProb 4	com.example.assistente_me	D E: 2.366244E-
2025-05-20 16:42:40.289 17319-17319 PredictionProb 1.5501396E-6	com.example.assistente_me	D F:
2025-05-20 16:42:40.289 17319-17319 PredictionProb 0.027249664	com.example.assistente_me	D G:
2025-05-20 16:42:40.290 17319-17319 PredictionProb 0.021738047	com.example.assistente_me	D H:
2025-05-20 16:42:40.290 17319-17319 PredictionProb	com.example.assistente_me	D I: 8.192843E-8
2025-05-20 16:42:40.290 17319-17319 PredictionProb 5	com.example.assistente_me	D J: 2.6528489E-
2025-05-20 16:42:40.290 17319-17319 PredictionProb 1.5514204E-4	com.example.assistente_me	D K:
2025-05-20 16:42:40.290 17319-17319 PredictionProb	com.example.assistente_me	D L: 7.81053E-7
2025-05-20 16:42:40.290 17319-17319 PredictionProb	com.example.assistente_me	D M: 0.21650913
2025-05-20 16:42:40.290 17319-17319 PredictionProb	com.example.assistente_me	D N: 0.03723592
2025-05-20 16:42:40.290 17319-17319 PredictionProb 0.0029429307	com.example.assistente_me	D O:
2025-05-20 16:42:40.290 17319-17319 PredictionProb 0.0015264346	com.example.assistente_me	D P:
2025-05-20 16:42:40.291 17319-17319 PredictionProb 0.013027788	com.example.assistente_me	D Q:
2025-05-20 16:42:40.291 17319-17319 PredictionProb 0.023011055	com.example.assistente_me	D R:
2025-05-20 16:42:40.291 17319-17319 PredictionProb 4	com.example.assistente_me	D S: 6.352297E-
2025-05-20 16:42:40.291 17319-17319 PredictionProb 7	com.example.assistente_me	D T: 6.381411E-
2025-05-20 16:42:40.291 17319-17319 PredictionProb 4	com.example.assistente_me	D U: 8.270617E-
2025-05-20 16:42:40.291 17319-17319 PredictionProb 2.7071576E-6	com.example.assistente_me	D V:
2025-05-20 16:42:40.291 17319-17319 PredictionProb 0.0015090065	com.example.assistente_me	D W:
2025-05-20 16:42:40.291 17319-17319 PredictionProb 5	com.example.assistente_me	D X: 8.839727E-
2025-05-20 16:42:40.291 17319-17319 PredictionProb 0.009354088	com.example.assistente_me	D Y:
2025-05-20 16:42:40.291 17319-17319 PredictionProb 4.9849194E-5	com.example.assistente_me	D Z:
2025-05-20 16:48:15.706 17319-17319 PhoneWindow setVisibility: visibility = 4, Parent = ViewRoot{14ea0f5 com.example.assistente_me/com.example.assistente_me.MainActivity,ident = 0}, this = DecorView@ee2a1df[MainActivity]	com.example.assistente_me	V DecorView

LogCat de la predicción


```

package com.example.assistente_me

import android.graphics.Bitmap
import android.graphics.Canvas
import com.example.assistente_me.ui.screens.DrawingScreen
import com.example.assistente_me.ui.screens.MainScreen

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.runtime.*
import androidx.compose.ui.geometry.Offset
import kotlin.math.max
import android.util.Log
import android.widget.Toast

class MainActivity : ComponentActivity() {

    private lateinit var classifier: LetterClassifier

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        classifier = LetterClassifier(assets, this)

        setContent {
            var prediction by remember { mutableStateOf("") }
            var showDrawingScreen by remember { mutableStateOf(false) }

            if (showDrawingScreen) {
                DrawingScreen(
                    onBack = { showDrawingScreen = false },
                    prediction = prediction,
                    onPredict = { points ->
                        if (points.isEmpty()) {
                            prediction = ""
                        } else {
                            try {
                                prediction = classifier.classify(points)
                            } catch (e: Exception) {
                                Log.e("MainActivity", "Error clasificando:
${e.message}")
                                prediction = "?"
                                Toast.makeText(this@MainActivity, "Error en
clasificación", Toast.LENGTH_SHORT).show()
                            }
                        }
                    }
                )
            } else {
                MainScreen(
                    onJuguemosClick = { showDrawingScreen = true }
                )
            }
        }

        override fun onDestroy() {
            super.onDestroy()
            classifier.close()
        }
    }
}

```

```
private fun pointsToBitmap(points: List<Offset>, width: Int, height: Int):
Bitmap {
    val bitmap = Bitmap.createBitmap(width, height,
Bitmap.Config.ARGB_8888)
    val canvas = Canvas(bitmap)
    canvas.drawColor(android.graphics.Color.WHITE)

    val paint = android.graphics.Paint().apply {
        color = android.graphics.Color.BLACK
        strokeWidth = 30f // menos grueso
        isAntiAlias = true
        strokeCap = android.graphics.Paint.Cap.ROUND
    }

    if (points.isNotEmpty()) {
        var prev = points[0]
        for (i in 1 until points.size) {
            val current = points[i]
            canvas.drawLine(prev.x, prev.y, current.x, current.y, paint)
            prev = current
        }
    }
    return bitmap
}

private fun processBitmap(bitmap: Bitmap): FloatArray {
    val croppedBitmap = cropAndCenterBitmap(bitmap)
    val resizedBitmap = Bitmap.createScaledBitmap(croppedBitmap, 28, 28,
true)

    val floatArray = FloatArray(28 * 28)
    for (y in 0 until 28) {
        for (x in 0 until 28) {
            val pixel = resizedBitmap.getPixel(x, y)
            val r = (pixel shr 16) and 0xFF
            val g = (pixel shr 8) and 0xFF
            val b = pixel and 0xFF
            val gray = (r + g + b) / 3f
            floatArray[y * 28 + x] = 1.0f - (gray / 255f)
        }
    }
    return floatArray
}

private fun cropAndCenterBitmap(original: Bitmap): Bitmap {
    val width = original.width
    val height = original.height
    var minX = width
    var minY = height
    var maxX = 0
    var maxY = 0

    for (y in 0 until height) {
        for (x in 0 until width) {
            val pixel = original.getPixel(x, y)
            val r = (pixel shr 16) and 0xFF
            val g = (pixel shr 8) and 0xFF
            val b = pixel and 0xFF
            val gray = (r + g + b) / 3
            if (gray < 200) {
                if (x < minX) minX = x
                if (y < minY) minY = y
            }
        }
    }
    val croppedWidth = width - minX + maxX
    val croppedHeight = height - minY + maxY
    val croppedBitmap = Bitmap.createBitmap(original, maxX, minY, croppedWidth,
croppedHeight)
    val centeredBitmap = Bitmap.createScaledBitmap(croppedBitmap, width, height,
true)
    return centeredBitmap
}
```

```

        if (x > maxX) maxX = x
        if (y > maxY) maxY = y
    }
}

if (minX >= maxX || minY >= maxY) {
    // Si no se detecta trazo negro, devolvemos bitmap blanco 28x28
    return Bitmap.createBitmap(28, 28, Bitmap.Config.ARGB_8888).apply {
        eraseColor(android.graphics.Color.WHITE)
    }
}

val cropped = Bitmap.createBitmap(original, minX, minY, maxX - minX,
maxY - minY)

val size = max(cropped.width, cropped.height)
val output = Bitmap.createBitmap(size, size, Bitmap.Config.ARGB_8888)
val canvas = Canvas(output)
canvas.drawColor(android.graphics.Color.WHITE)
val left = (size - cropped.width) / 2f
val top = (size - cropped.height) / 2f
canvas.drawBitmap(cropped, left, top, null)

return output
}
}

```

Código 3. MainActivity

```

package com.example.assistente_me

import android.content.Context
import android.content.res.AssetManager
import android.graphics.Bitmap
import android.graphics.Canvas
import android.graphics.Color
import android.graphics.Paint
import android.graphics.Path
import android.util.Log
import androidx.compose.ui.geometry.Offset
import org.tensorflow.lite.Interpreter
import java.io.File
import java.io.FileInputStream
import java.io.FileOutputStream
import java.nio.MappedByteBuffer
import java.nio.channels.FileChannel

class LetterClassifier(
    assetManager: AssetManager,
    private val context: Context, // Agregado contexto para guardar
    archivos
    modelPath: String = "letter_model.tflite"
) {

    private var tflite: Interpreter = Interpreter(loadModelFile(assetManager,
modelPath))

    private fun loadModelFile(assetManager: AssetManager, modelPath: String):
MappedByteBuffer {
        val fileDescriptor = assetManager.openFd(modelPath)
    }
}

```

```

        val inputStream = FileInputStream(fileDescriptor.fileDescriptor)
        val fileChannel = inputStream.channel
        val startOffset = fileDescriptor.startOffset
        val declaredLength = fileDescriptor.declaredLength
        return fileChannel.map(FileChannel.MapMode.READ_ONLY, startOffset,
declaredLength)
    }

    private fun normalizePoints(points: List<Offset>, size: Float):
List<Offset> {
        val minX = points.minOf { it.x }
        val maxX = points.maxOf { it.x }
        val minY = points.minOf { it.y }
        val maxY = points.maxOf { it.y }

        val width = maxX - minX
        val height = maxY - minY

        if (width == 0f || height == 0f) {
            return points.map { Offset(size / 2f, size / 2f) }
        }

        val scaleX = size / width
        val scaleY = size / height
        val scale = minOf(scaleX, scaleY) * 0.8f // Usa menos zoom para no
cortar letras

        val offsetX = (size - width * scale) / 2f
        val offsetY = (size - height * scale) / 2f

        return points.map {
            Offset(
                x = (it.x - minX) * scale + offsetX,
                y = (it.y - minY) * scale + offsetY
            )
        }
    }

    private fun createBitmapFromPoints(points: List<Offset>): Bitmap {
        val bitmapSize = 280
        val bitmap = Bitmap.createBitmap(bitmapSize, bitmapSize,
Bitmap.Config.ARGB_8888)
        val canvas = Canvas(bitmap)
        canvas.drawColor(Color.WHITE)

        val paint = Paint().apply {
            color = Color.BLACK
            strokeWidth = 30f
            style = Paint.Style.STROKE
            strokeCap = Paint.Cap.ROUND
            strokeJoin = Paint.Join.ROUND
            isAntiAlias = true
        }

        val normalizedPoints = normalizePoints(points, bitmapSize.toFloat())
        val path = Path()
        path.moveTo(normalizedPoints[0].x, normalizedPoints[0].y)
        for (point in normalizedPoints.drop(1)) {
            path.lineTo(point.x, point.y)
        }
        canvas.drawPath(path, paint)
    }

```

```

        return Bitmap.createScaledBitmap(bitmap, 28, 28, true)
    }

    private fun softmax(logits: FloatArray): FloatArray {
        val maxLogit = logits.maxOrNull() ?: 0f
        val expScores = logits.map { Math.exp((it - maxLogit).toDouble()) }
        val sumExp = expScores.sum()
        return expScores.map { (it / sumExp).toFloat() }.toFloatArray()
    }

    // Nueva función para guardar bitmap en almacenamiento interno para debug
    private fun saveBitmapDebug(bitmap: Bitmap, fileName: String =
"debug_letter.png") {
        val picturesDir = context.getExternalFilesDir(null) // Carpeta
accesible
        val file = File(picturesDir, fileName)

        try {
            FileOutputStream(file).use { out ->
                bitmap.compress(Bitmap.CompressFormat.PNG, 100, out)
            }
            Log.d("BitmapDebug", "Imagen guardada en: ${file.absolutePath}")
        } catch (e: Exception) {
            Log.e("BitmapDebug", "Error guardando imagen: ${e.message}")
        }
    }

    fun classify(points: List<Offset>): String {
        if (points.isEmpty()) return "Dibuja algo primero"
        if (points.size == 1) return "Dibuja una línea o forma"

        val bitmap = createBitmapFromPoints(points)

        // Guardar bitmap para debug
        saveBitmapDebug(bitmap)

        val input = Array(1) { Array(28) { Array(28) { FloatArray(1) } } }

        for (y in 0 until 28) {
            for (x in 0 until 28) {
                val pixel = bitmap.getPixel(x, y)
                val r = Color.red(pixel)
                val g = Color.green(pixel)
                val b = Color.blue(pixel)
                val gray = (r + g + b) / 3f
                val value = gray / 255f // <- SIN invertir
                input[0][y][x][0] = value
            }
        }

        val output = Array(1) { FloatArray(26) }
        tflite.run(input, output)

        val probabilities = softmax(output[0])

        val letters = ('A'..'Z').toList()
        for (i in probabilities.indices) {
            Log.d("PredictionProb", "${letters[i]}: ${probabilities[i]}")
        }

        val predIndex = probabilities.indices.maxByOrNull { probabilities[it] }

```

```
?: -1
    if (predIndex == -1) return "No se pudo predecir"

    return letters[predIndex].toString()
}

fun close() {
    tflite.close()
}
}
```

Código 4. LetterClassifier

Conclusión

El proyecto Asistente Mágico de Escritura cumplió con el propósito de desarrollar una herramienta interactiva que apoya el aprendizaje de la escritura en niños de nivel preescolar y primaria. A través de un sistema de reconocimiento de escritura basado en inteligencia artificial, se logró identificar y corregir errores comunes en el trazo de letras y números, brindando retroalimentación inmediata de manera visual y auditiva.

Uno de los principales objetivos era convertir el proceso de aprendizaje en una experiencia motivadora, y se logró mediante la integración de elementos lúdicos como sonidos, animaciones y recompensas que incentivaron la práctica constante. Además, la accesibilidad y facilidad de uso de la aplicación permitieron que los niños interactuaran con la herramienta de manera intuitiva, fomentando su autonomía y mejorando su escritura progresivamente.

El desarrollo del sistema incorporó técnicas de procesamiento de imágenes y modelos de aprendizaje automático entrenados con datos reales, garantizando que el reconocimiento de escritura fuera preciso y funcional. Gracias a las pruebas realizadas, se verificó la efectividad del modelo y se realizaron ajustes que optimizaron su rendimiento para mejorar la experiencia del usuario.

Con el lanzamiento de la aplicación, el Asistente Mágico de Escritura se consolida como una alternativa educativa innovadora que combina tecnología y pedagogía, proporcionando una solución efectiva para mejorar la enseñanza de la escritura infantil. A partir de ahora, su implementación permitirá seguir explorando mejoras y ajustes basados en la interacción real de los niños con la herramienta, asegurando su impacto positivo en el desarrollo cognitivo y motor de sus usuarios.

Bibliografías

Carballido, B. V., & Carballido, B. V. (2024, 30 septiembre). *Alfabetización en Inteligencia Artificial con Teachable Machine* | DF Diario Feminista. DF Diario Feminista | Medio Feminista de Publicación Diaria. <https://eldiariofeminista.info/2024/09/30/alfabetizacion-en-inteligencia-artificial-con-teachable-machine/>

Machine Learning for Kids. (s. f.). <https://machinelearningforkids.co.uk/>

Teachable machine. (s. f.). https://teachablemachine.withgoogle.com/?utm_source=chatgpt.com

Arqueros, V. (2024, 23 septiembre). *5 Herramientas de inteligencia artificial para docentes*. Educrea. https://educrea.cl/5-herramientas-de-inteligencia-artificial-para-docentes/?utm_source=chatgpt.com

Pittawat Taveekitworachai. (2023). *Letter_recognition*. Huggingface.co. [HTTPS://HUGGINGFACE.CO/DATASETS/PITTAWAT/LETTER_RECOGNITION](https://huggingface.co/datasets/pittawat/letter_recognition)